

Diffusion Policies as an Expressive Policy Class for Offline Reinforcement Learning

Zhichen Zhong, Boyang Xia, Jiawen Dai
School of Information Science and Technology, ShanghaiTech University, Shanghai City, China
{zhongzhch2023, xiaby2023, daijw2023}@shanghaitech.edu.cn

Background

- Offline Reinforcement Learning (Offline RL or Batch RL) presents a significant paradigm for learning decision-making policies in a wide array of real-world scenarios. A primary challenge in offline RL is **distributional shift**: policies trained naively on a static dataset may learn to exploit errors in the value function estimates for out-of-distribution (OOD) actions, leading to poor performance when deployed.
- The ICLR 2023 paper[1], argues that many policy regularization methods suffer because the policy classes they use (e.g., Gaussian policies) are not expressive enough to accurately model complex, potentially multimodal, behavior policies often found in offline datasets. This can lead to improper regularization and suboptimal solutions. Diffusion models, known for their ability to model complex distributions, offer a promising alternative.
- Core Idea: Diffusion Q-Learning (Diffusion-QL)** The experimental results in the paper robustly demonstrate that the outstanding expressiveness of the diffusion policy, coupled with the effective integration of behavior cloning (as an implicit regularization) and Q-learning guidance, are the key contributors to Diffusion-QL's state-of-the-art performance across numerous D4RL benchmark tasks.

Reproduction of experiments in the paper

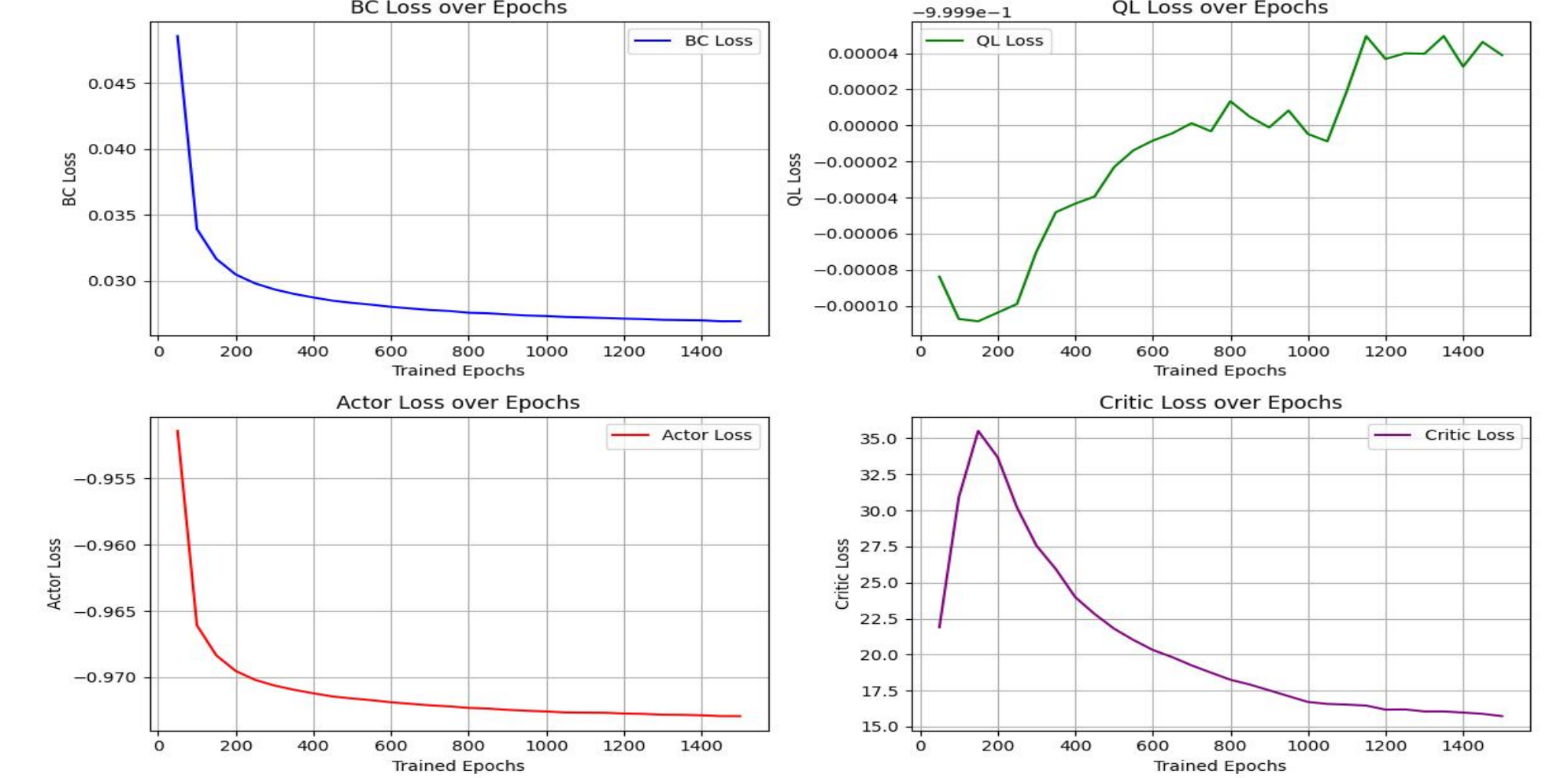


Figure 11: Diffusion QL for walker2d-medium-expert-v2

Methods & Result

Behavior Cloning & Policy Improvement & Extended Investigation

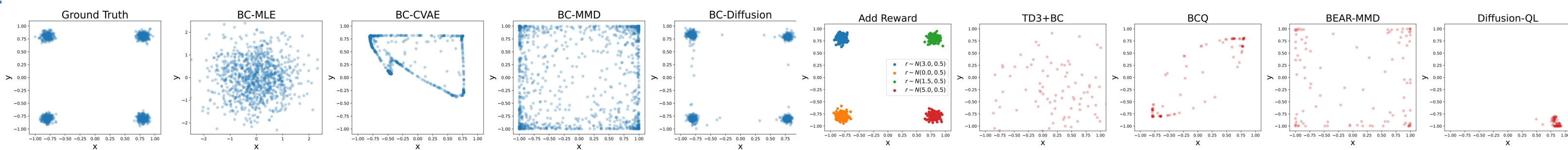


Figure 1: Equal Gaussian Mixture Distribution: BC

Figure 2: Equal Gaussian Mixture Distribution: QL

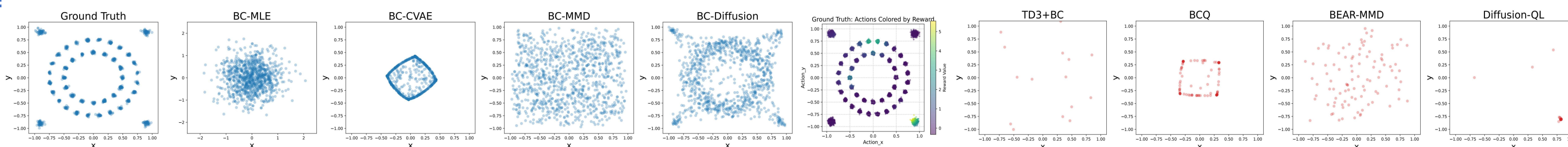


Figure 3: Complex distribution 1: Increased Modal Complexity

Figure 4: Complex distribution 1: Increased Modal Complexity

- BC-MLE, BC-CAVE, BC-MMD, BC-Diffusion; TD3+BC, BCQ, BEAR-MMD, Diffusion-QL
- Performance of BC:** The policy of BC-MLE is limited to a single mode. The CVAE model even fails to exhibit mode-covering behavior, excluding the corner samples. The tanh Gaussian policy optimized under MMD fails to capture the true distribution. Diffusion seems to be the best among the four, but it fails to distinguish between the inner ring and outer ring
- Performance of QL:** As expected, TD3+BC, BCQ, and BEAR-MMD fail to converge to high reward regions since they cannot capture the true policy distribution. Diffusion QL, however, performs well with the guidance of Q-Learning, even though it fails to accurately capture the ground truth.

Other Toy Bandit Examples & The influence of N:

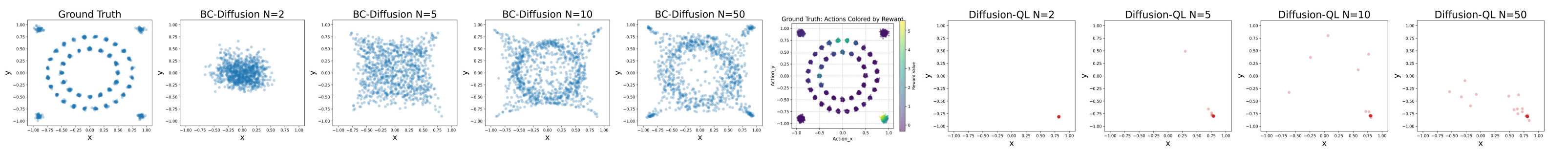


Figure 5: Complex distribution 1: Modify N BC

Figure 6: Complex distribution 1: Modify N QL

- The influence of N:** As N increases, the scattered points have decreased, and more data points are increasingly concentrated in the original distribution locations. And a complex policy distribution takes tigger N to converge.

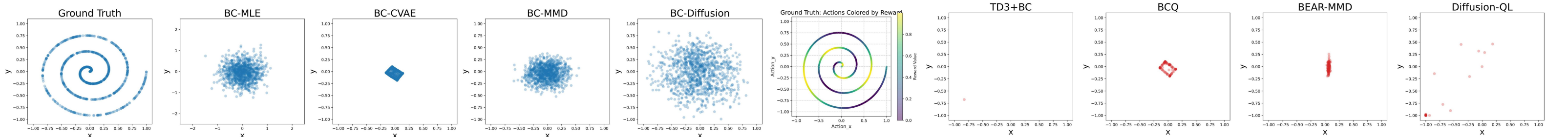


Figure 7: Complex distribution 2: Mosquito coil shape

Figure 8: Complex distribution 2: Mosquito coil shape

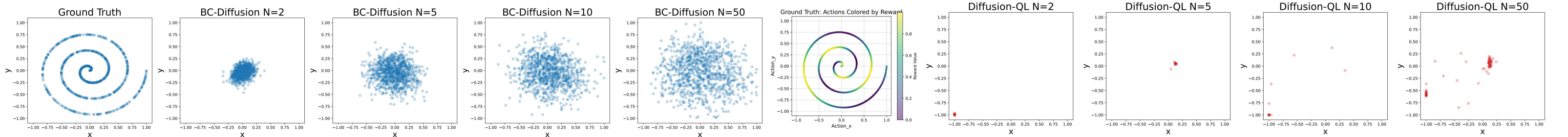


Figure 9: Complex distribution 2: Modify N BC

Figure 10: Complex distribution 2: Modify N QL

- For complex policy,** simply increasing the size of N does not lead to a significant improvement in performance

Our Innovations

Building upon the ability of EDP and DPPO to address distinct limitations of Diffusion QL, our primary innovation is a synergistic two-stage framework that integrates these algorithms. This approach is structured as follows:

- Offline Pre-training with EDP:** In the initial phase, EDP is employed for robust policy and value (or Q-value) function pre-training using the available offline dataset. For the purpose of this combined framework, EDP is utilized purely for its offline learning capabilities, without a subsequent online improvement phase by EDP itself.
- Online Fine-tuning with DPPO:** Following the EDP pre-training, DPPO is deployed for online policy refinement through interaction with the environment.

This methodology aims to capitalize on EDP's effectiveness in leveraging offline data to establish a strong initial policy and value estimation, which DPPO then further enhances and adapts through efficient online interaction and policy gradient updates.

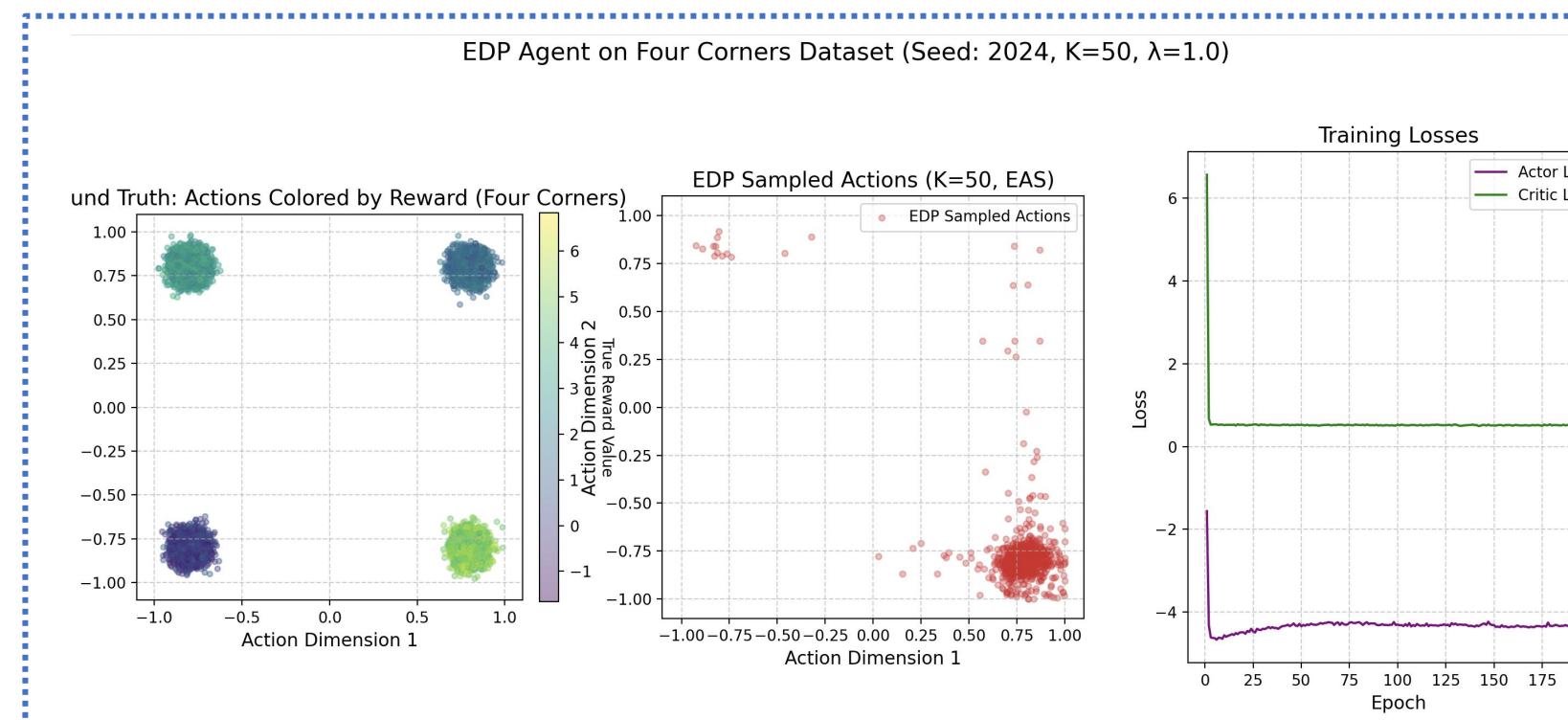


Figure 12: Equal Gaussian Mixture Distribution

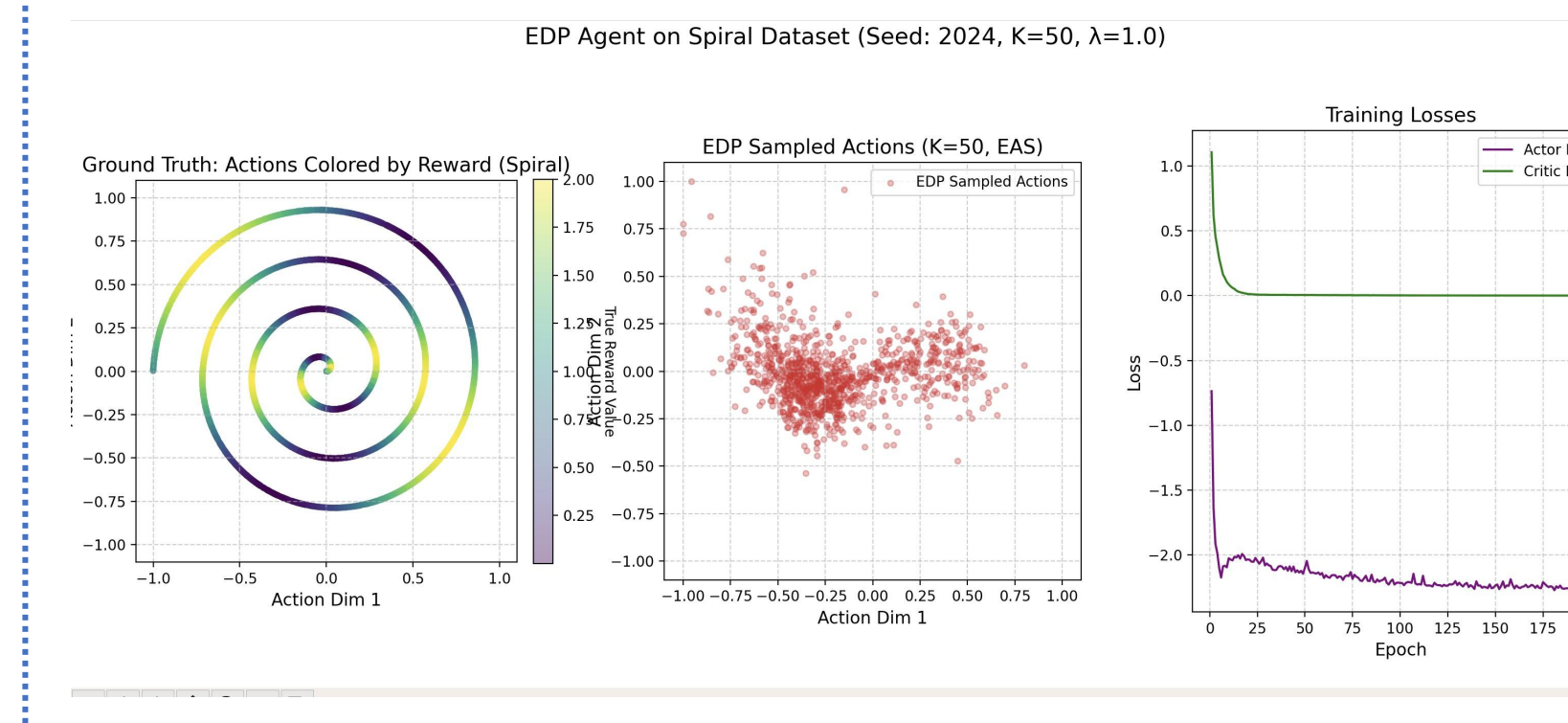


Figure 14: Mosquito coil shape

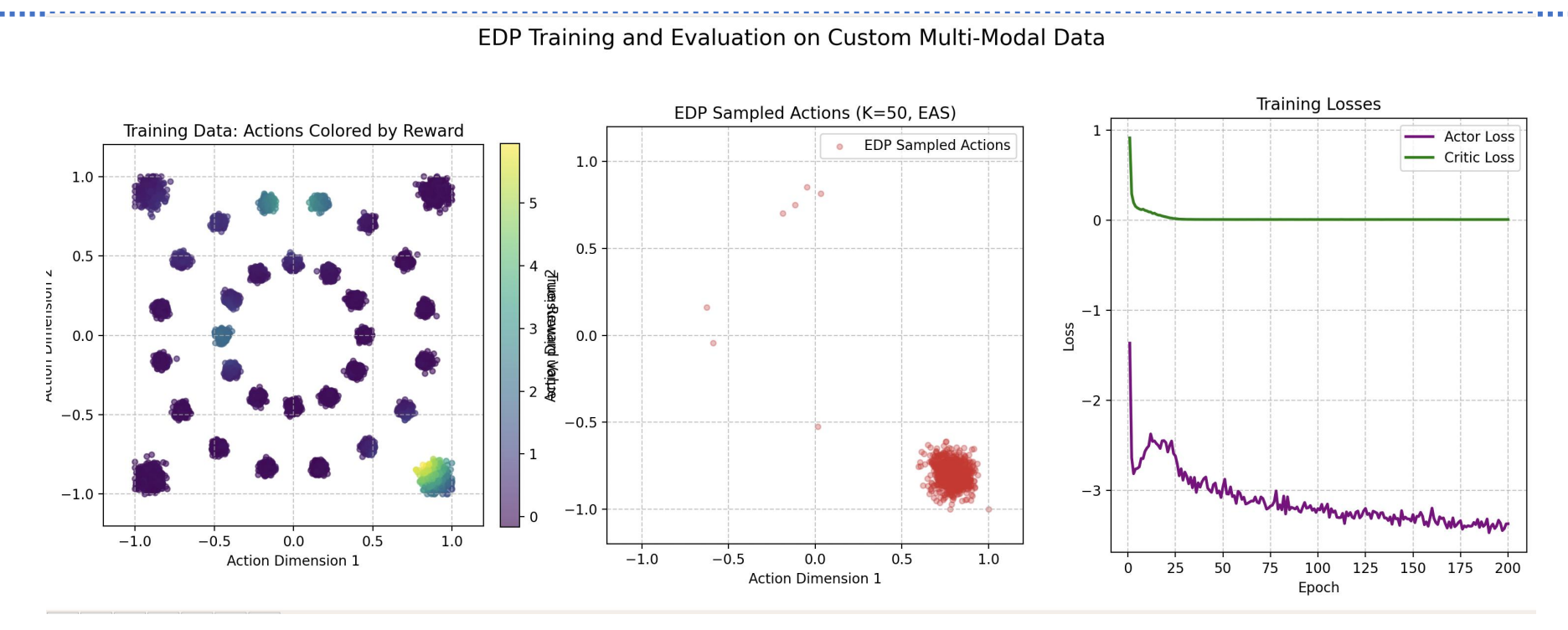


Figure 13: Increased Modal Complexity

The Efficient Diffusion Policy (EDP) tackles two major drawbacks of Diffusion-QL:

- High computational costs from full Markov chain propagation during training.
- Incompatibility with likelihood-based RL algorithms due to diffusion models' intractable likelihood.

Pseudocode

Algorithm 1 Diffusion Q-learning

```
Initialize policy network  $\pi_\theta$ , critic networks  $Q_{\phi_1}$  and  $Q_{\phi_2}$ , and target networks  $\pi_{\theta'}$ ,  $Q_{\phi'_1}$  and  $Q_{\phi'_2}$ 
for each iteration do
  Sample transition mini-batch  $\mathcal{B} = \{(s_t, a_t, r_t, s_{t+1})\} \sim \mathcal{D}$ .
  # Q-value function learning
  Sample  $a_{t+1}^0 \sim \pi_{\theta'}(a_{t+1} | s_{t+1})$  by Equation (1).
  Update  $Q_{\phi_1}$  and  $Q_{\phi_2}$  by Equation (4). (max Q backup by Kumar et al. (2020) could be added)
  # Policy learning
  Sample  $a_t^0 \sim \pi_{\theta}(a_t | s_t)$  by Equation (1).
  Update policy by minimizing Equation (3).
  # Update target networks
   $\theta' = \rho\theta' + (1 - \rho)\theta$ ,  $\phi'_i = \rho\phi'_i + (1 - \rho)\phi_i$  for  $i = \{1, 2\}$ .
end for
```

Reference

- [1] Wang, Zhendong, Jonathan J. Hunt, and Mingyuan Zhou. "Diffusion policies as an expressive policy class for offline reinforcement learning." arXiv preprint arXiv:2208.06193 (2022)
- [2] Kang, Bingyi, et al. "Efficient diffusion policies for offline reinforcement learning." Advances in Neural Information Processing Systems 36 (2023): 67195-67212
- [3] Ren, Allen Z., Justin Lidard, Lars L. Ankile, Anthony Simeonov, Pulkit Agrawal, Anirudha Majumdar, Benjamin Burchfiel, Hongkai Dai, and Max Simchowitz. "Diffusion Policy Policy Optimization." arXiv preprint arXiv:2409.00588v3 (2024)

Algorithm 1: Reinforcement Guided Diffusion Policy

```
Input:  $I, \lambda, \eta, \mathcal{D}, \epsilon_\theta(a^k, k; s), Q_\phi(s, a)$ 
Output:  $\theta, \phi$ 
for  $i = 0, \dots, I$  do
  // Sample a batch of data
   $\{(s, a, s')\} \sim \mathcal{D}$ 
  // Sample next actions with DPM-Solver
   $a' \sim \pi_\theta(\cdot | s)$ 
   $\phi \leftarrow \phi - \eta \nabla_\phi L_{TD}(\phi)$ 
  // Action approximation
   $\hat{a}^0 = \frac{1}{\sqrt{\alpha^k}} a^k - \frac{\sqrt{1-\alpha^k}}{\sqrt{\alpha^k}} \epsilon_\theta(a^k, k; s)$ 
   $\theta \leftarrow \theta - \eta \nabla_\theta (L_{diff}(\theta) + \lambda L_\pi(\theta))$ 
```

Algorithm 2: Energy-based Action Selection

```
Input: number of actions:  $N, \pi_\theta(a|s), Q_\phi(s, a)$ 
Output:  $a$ 
 $a_i \sim \pi_\theta(a|s) \quad i = 1, \dots, N$ 
 $a = \text{categorical\_sample}(\{a_i\}, \{e^{Q_\phi(s, a_i)}\})$ 
```

Algorithm 1 DPPO

```
1: Pre-train diffusion policy  $\pi_\theta$  with offline dataset  $\mathcal{D}_{\text{off}}$  using BC loss  $\mathcal{L}_{BC}(\theta)$  Eq. (A1).
2: Initialize value function  $V_\phi$ .
3: for iteration = 1, 2, ... do
4:   Initialize rollout buffer  $\mathcal{D}_{\text{tr}}$ .
5:    $\pi_{\theta_{\text{old}}} = \pi_\theta$ .
6:   for environment = 1, 2, ...,  $N$  in parallel do
7:     Initialize state  $\tilde{s}_{i(0,K)} = (s_0, a_0^k)$  in  $\mathcal{M}_{\text{DP}}$ .
8:     for environment step  $t = 1, \dots, T$ , denoising step  $k = K - 1, \dots, 0$  do
9:       Sample the next denoised action  $\tilde{a}_{i(t,k)} = a_t^k \sim \pi_{\theta_{\text{old}}}$ .
10:      if  $k = 0$  then
11:        Run  $a_t^0$  in the environment and observe  $\tilde{R}_{i(t,0)}$  and  $\tilde{s}_{i(t+1,K)}$ .
12:      else
13:        Set  $\tilde{R}_{i(t,k)} = 0$  and  $\tilde{s}_{i(t,k-1)} = (s_t, a_t^k)$ .
14:        Add  $(k, \tilde{s}_{i(t,k)}, \tilde{a}_{i(t,k)}, \tilde{R}_{i(t,k)})$  to  $\mathcal{D}_{\text{tr}}$ .
15:      Compute advantage estimates  $A_{i(t,k)}^{\pi_{\theta_{\text{old}}}}(\tilde{s}_{i(t,k=0)}, \tilde{a}_{i(t,k=0)})$  for  $\mathcal{D}_{\text{tr}}$  using GAE Eq. (A2).
16:      for update = 1, 2, ..., num_update do
17:        for minibatch = 1, 2, ...,  $B$  do
18:          Sample  $(k, \tilde{s}_{i(t,k)}, \tilde{a}_{i(t,k)}, \tilde{R}_{i(t,k)})$  and  $A_{i(t,k)}^{\pi_{\theta_{\text{old}}}}(\tilde{s}_{i(t,k)}, \tilde{a}_{i(t,k)})$  from  $\mathcal{D}_{\text{tr}}$ .
19:          Compute denoising-discounted advantage  $\hat{A}_{i(t,k)} = \gamma^k_{\text{DENOISE}} A_{i(t,k)}^{\pi_{\theta_{\text{old}}}}(\tilde{s}_{i(t,0)}, \tilde{a}_{i(t,0)})$ .
20:          Optimize  $\pi_\theta$  using policy gradient loss  $\mathcal{L}_\theta$  Eq. (A3).
21:          Optimize  $V_\phi$  using value loss  $\mathcal{L}_\phi$  Eq. (A4).
22: return converged policy  $\pi_\theta$ .
```